Solution: Meeting Rooms II

Problem Statement

Given a list of time intervals during which meetings are scheduled, determine the minimum number of meeting rooms that are required to ensure that none of the meetings overlap in time.

Examples

1. **Example 1:**
   - **Input:** `[[10, 15], [20, 25], [30, 35]]`
   - **Expected Output:** `1`
   - **Justification:** There are no overlapping intervals in the given list. So, only 1 meeting room is enough for all the meetings.
2. **Example 2:**
   - **Input:** `[[10, 20], [15, 25], [24, 30]]`
   - **Expected Output:** `2`
   - **Justification:** The first and second intervals overlap, and the second and third intervals overlap as well. So, we need 2 rooms.
3. **Example 3:**
   - **Input:** `[[10, 20], [20, 30]]`
   - **Expected Output:** `1`
   - **Justification:** The end time of the first meeting is the same as the start time of the second meeting. So, one meeting can be scheduled right after the other in the same room.

Solution

To determine the minimum number of rooms required to host the meetings without any time overlap, our approach first involves sorting all meeting intervals based on their start times. This sorting allows us to sequentially evaluate meetings in the order they start. We then utilize a priority queue (min-heap) to keep track of end times of the meetings currently taking place. This queue helps in efficiently determining the earliest ending meeting. By sequentially examining each meeting and comparing its start time to the earliest ending time from the heap, we can decide if a new room is needed or if an existing room can be reused.

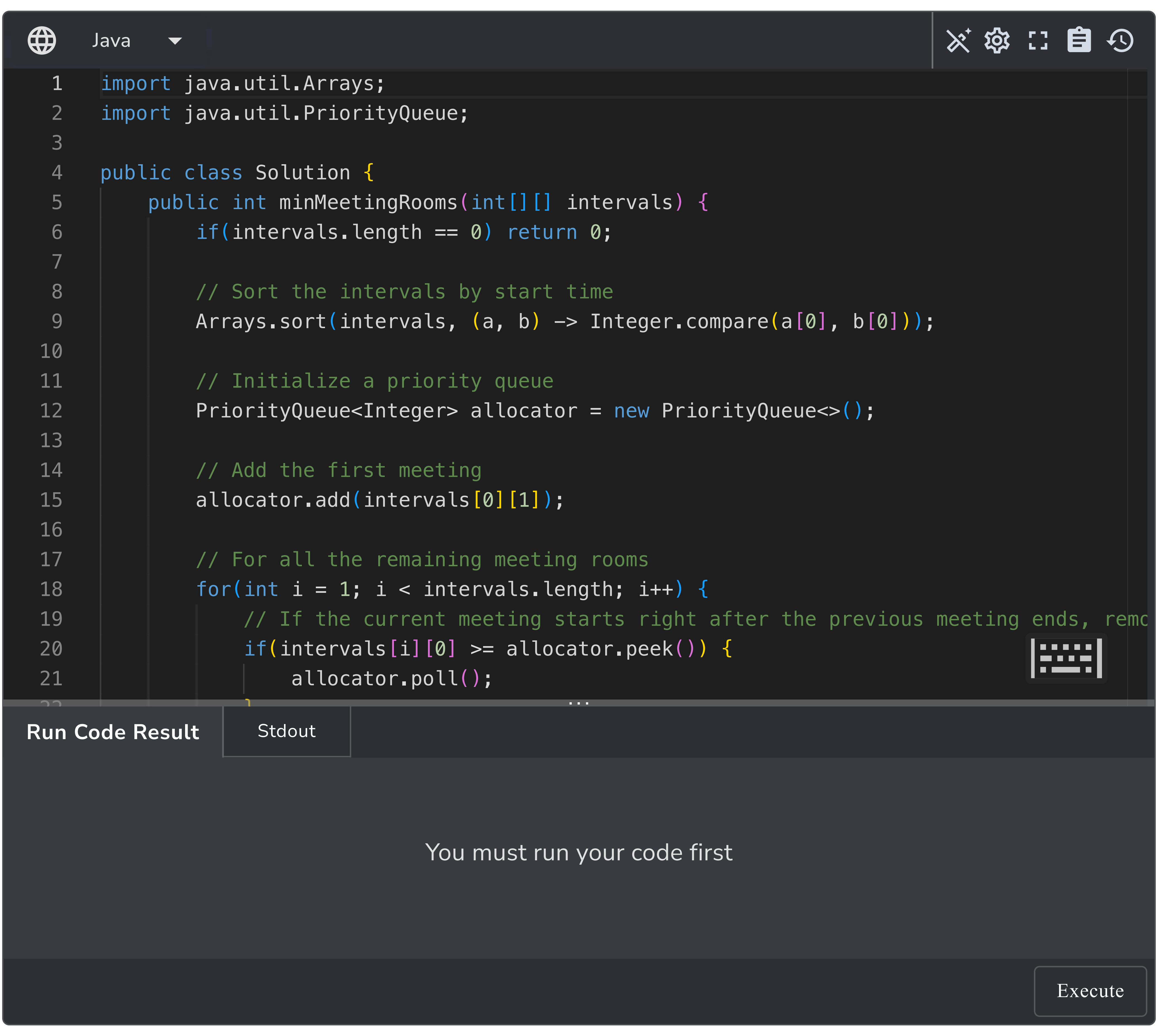
1. **Sorting**: Begin by sorting all intervals based on their start times. This enables us to sequentially check for overlapping intervals.
2. **Priority Queue**: A priority queue (min-heap) is used to monitor end times of meetings that are currently in session. If the start time of the next meeting is less than the smallest end time (i.e., the top of the priority queue), it indicates we require another room.
3. **Reusing Rooms**: Once we have checked a meeting and it doesn't overlap with the smallest end time, this means that the meeting room used by the meeting with the smallest end time can be reassigned. Therefore, we remove the earliest ending meeting from the priority queue.
4. **Counting Rooms**: At any given time, the size of the priority queue reflects the number of rooms in use. This can be used to deduce the minimum number of rooms needed up to that point.

Algorithm Walkthrough

Given an input `[[10, 20], [15, 30], [25, 40]]`:

- First, sort the intervals: `[[10, 20], [15, 30], [25, 40]]` (in this case, the intervals are already sorted).
- Initialize a priority queue. Add the end time of the first meeting to the queue.
- For the next interval `[15, 30]`, the start time 15 is less than the top of the queue (i.e., 20). Hence, we need another room. Add 30 to the priority queue.
- For the next interval `[25, 40]`, the start time 25 is greater than the top of the queue (i.e., 20). So, we can use the room which will be free at time 20. Remove 20 from the queue and add 40.
- The size of the priority queue at the end is 2, indicating 2 rooms are required.

Code



```java
import java.util.Arrays;
import java.util.PriorityQueue;

public class Solution {
    public int minMeetingRooms(int[][] intervals) {
        if(intervals.length == 0) return 0;

        // Sort the intervals by start time
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));

        // Initialize a priority queue
        PriorityQueue<Integer> allocator = new PriorityQueue<>();

        // Add the first meeting
        allocator.add(intervals[0][1]);

        // For all the remaining meeting rooms
        for(int i = 1; i < intervals.length; i++) {
            // If the current meeting starts right after the previous meeting ends, remo
            if(intervals[i][0] >= allocator.peek()) {
                allocator.poll();
```

Complexity Analysis

- **Time Complexity**: The time complexity of our algorithm is (O(N \log N)), where (N) is the number of intervals. This is because we're sorting the intervals once and then using priority queues to process them.
- **Space Complexity**: The space complexity is (O(N)) as we're storing all intervals in the worst case.

← Previous
Meeting Rooms II (Medium)
Next →
Encode And Decode Strings
☐ Mark as Completed